

Osmocom + nanoBTS

Building and running a cellular lab

Discovery, core setup, service tests and evidence

Mike Devaney / October 2026

The lab we will build

Four milestones, each with an observable result

01 **Connect the radio**

Discover the nanoBTS and connect it to OsmoBSC.

02 **Bring up the core**

See OML/RSL and core signalling working.

03 **Prove the services**

Make a call, deliver an SMS and establish a PDP session.

04 **Run a repeatable test**

Capture the result, then return to a known starting state.



Course Materials

Use my github repo…

git clone https://github.com/mike-rootworks/osmocom-configs



01

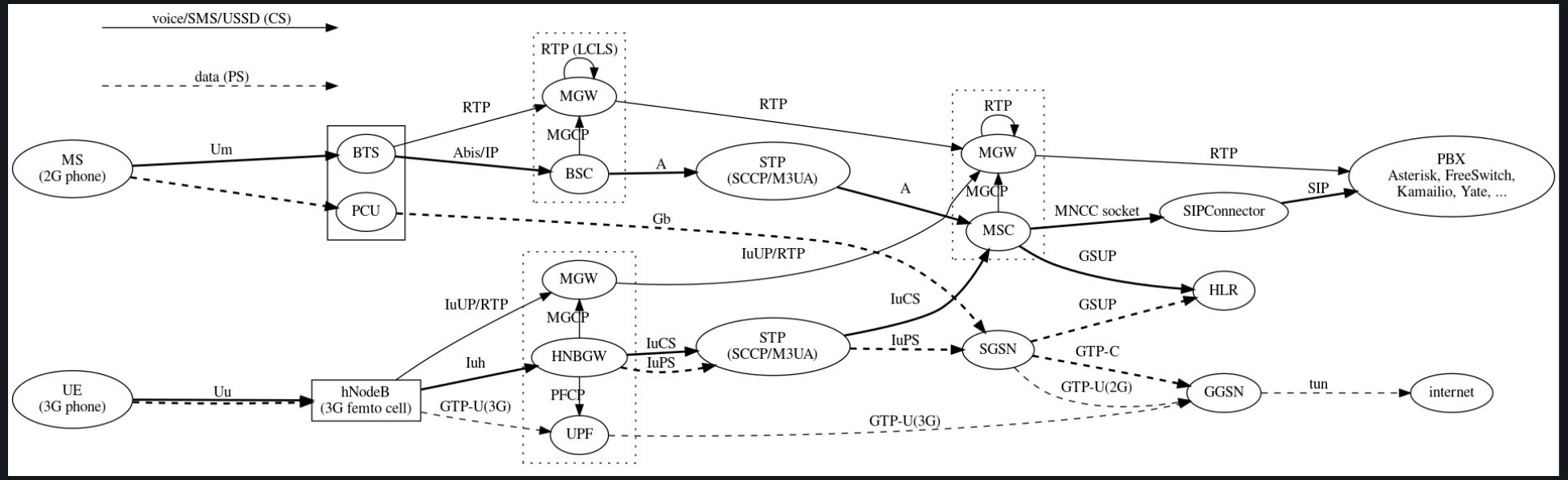
The network in the room

A 2G lab with voice, SMS and optional packet data



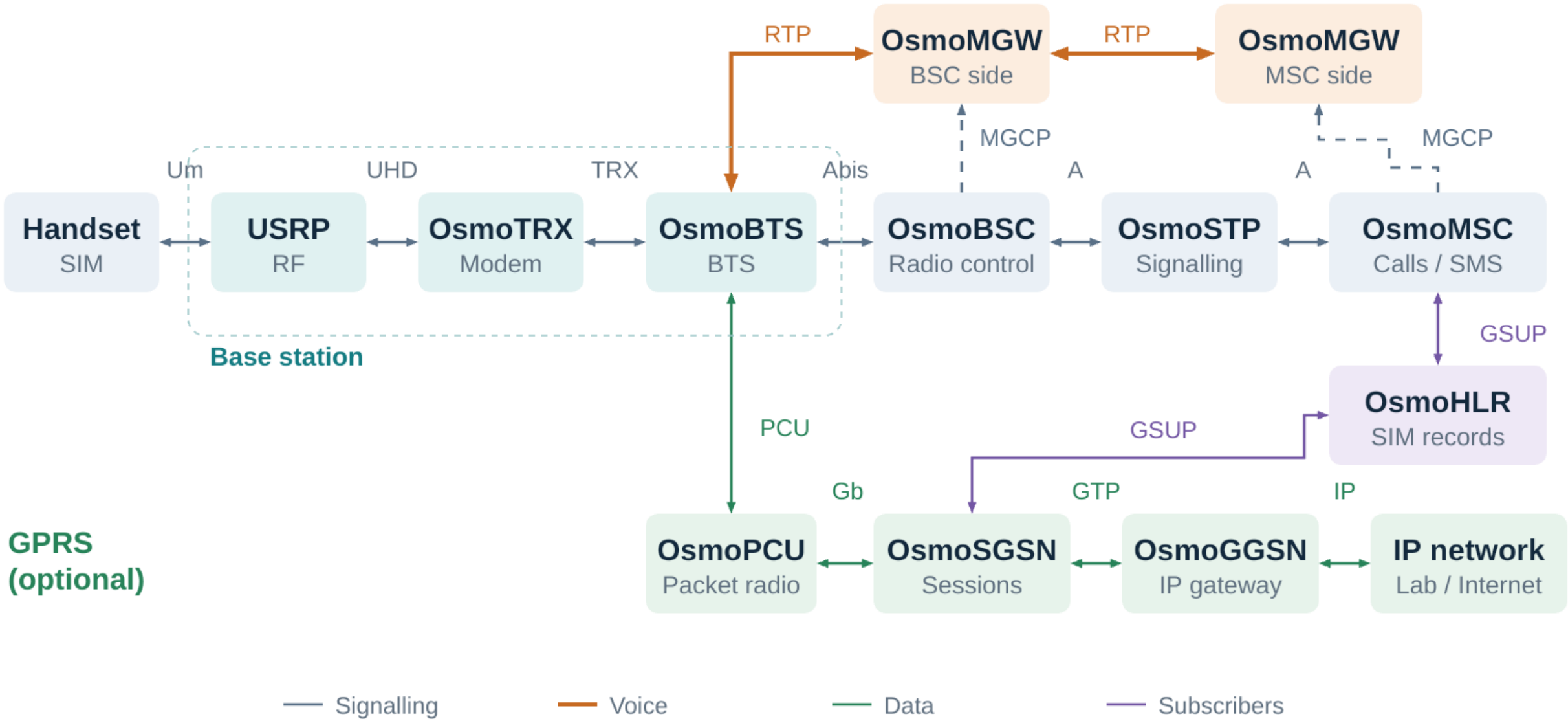
Wider 2G / 3G architecture

Context for the lab's place in the wider Osmocom ecosystem.

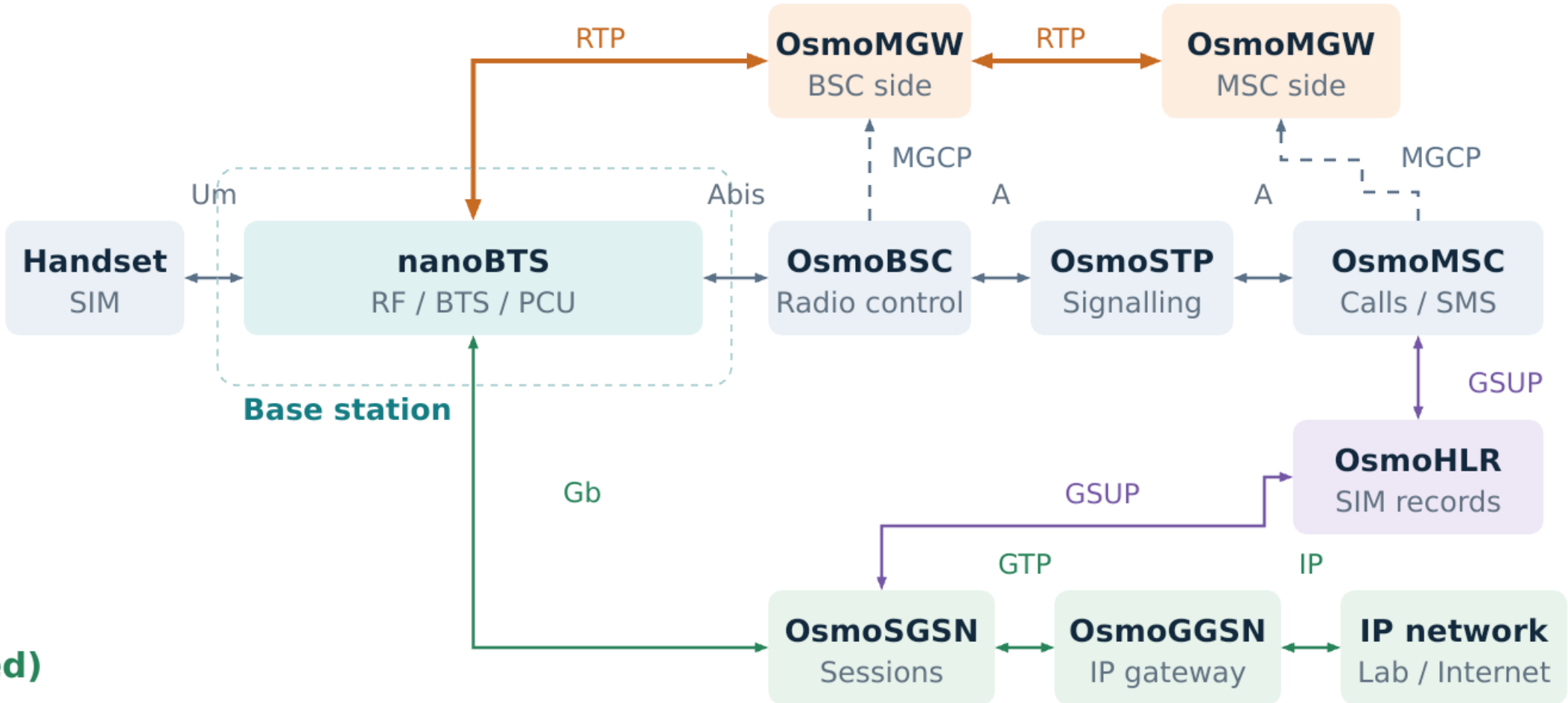


This workshop uses the 2G nanoBTS path. The 3G and external PBX paths are context.

Osmocom with a USRP



Osmocom with a nanoBTS



Component reference

Acronyms and responsibilities in one place

Component	Expanded name	Role
nanoBTS	Base Transceiver Station	Radio and BTS functions
OsmoBSC	Base Station Controller	Radio resource control
OsmoSTP	Signal Transfer Point	Routes core signalling
OsmoMSC	Mobile Switching Centre	Registration, calls and SMS
OsmoHLR	Home Location Register	Subscriber records and auth
OsmoMGW	Media Gateway	Voice media
OsmoSGSN	Serving GPRS Support Node	Packet mobility and sessions
OsmoGGSN	Gateway GPRS Support Node	Packet gateway to IP



Three paths to understand

The first troubleshooting question is which path failed.

Calls

BSC and MSC set up the call. OsmoMGW carries the audio as RTP.

SMS

OsmoMSC handles SMS through its SMSC. Packet data is not required.

GPRS / EDGE

The PCU connects to SGSN. GGSN exposes the device’s IP traffic to Linux.



Choosing the radio

A 165 DCS1800 unit is the course baseline.



Model and band

Read the label and record the firmware. A 165 model number alone does not identify the band.

Capability

Confirm packet-data support on the actual unit before the GPRS exercise.

GSM900, DCS1800 and PCS1900 are hardware variants, not selectable software profiles.

nanoBTS 165 — Antennas & Connectors



AUX — Alternate TX location
Used for stacked two-radio installations

NWL — Network Listen
Scans nearby cells; aids frequency calibration

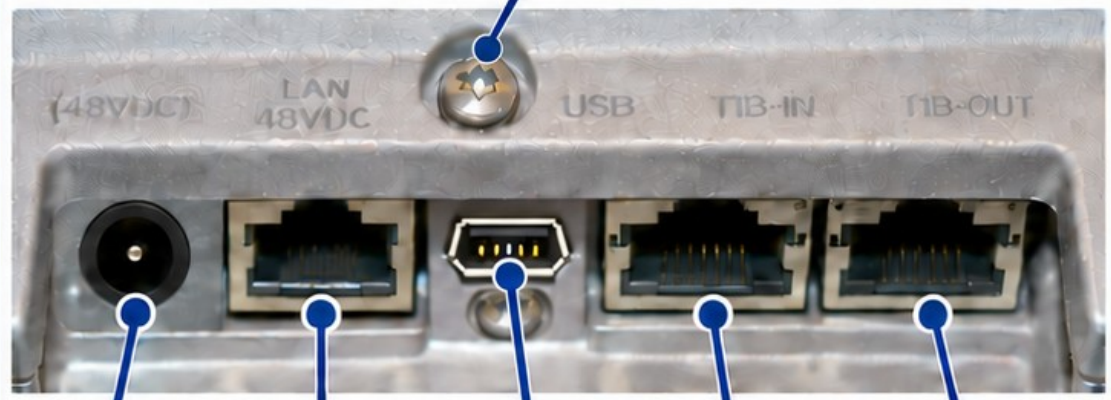
TX — Transmit
Sends GSM signals to phones

Status LED
Shows operating state

RX — Receive
Receives GSM signals from phones

Front • cover removed • ports downward

Bottom connector bay



Chassis bond
Bond chassis together in multi-radio setups

48V DC
Alternative power input

LAN / Ethernet
Link to OsmoBSC + 48V power

USB
Not used

TIB-IN
Timing / sync from previous BTS

TIB-OUT
Timing / sync to next BTS

TIB = Timing Interface Bus

Antenna fittings vary by 165 variant; 165E/F have no removable antennas.

nanoBTS 165 — LED Patterns

Read each strip from left to right; the pattern repeats.

COLOUR / PATTERN	LED ON / OFF OVER TIME	MEANING
Red Steady	 Continuously lit	Self-test failure / fatal software error
Red Fast blink	 0.3 s ON / 0.1 s OFF	Factory defaults applied (reset dongle)
Red / green Alternating • fast flash	 Alternating colours (schematic)	Unit not configured
Orange Slow flash	 1 s ON / 1 s OFF	Ethernet disconnected
Orange Fast flash	 0.5 s ON / 0.5 s OFF	Software download in progress
Orange Slow blink	 1.9 s ON / 0.1 s OFF	Waiting for management link
Orange Steady	 Continuously lit	Power-on self-test in progress
Green Fast flash	 0.5 s ON / 0.5 s OFF	Network Listen test in progress
Green / orange Alternating • slow blink	 Alternating colours (schematic)	Frequency calibration in progress
Green Slow flash	 1 s ON / 1 s OFF	Radio carrier not transmitting
Green Steady	 Continuously lit	**Operational**

Grey = LED off

Flash: equal ON / OFF

Blink: brief dark gaps

Higher-priority states override lower ones.

Sources: ip.access Installation Manual §2.5.9; Product Description Table 3. Alternating-colour timings are illustrative.

Exercise 1: identify the unit

Before changing a setting, record what you have.

01 Record model, band, serial number and firmware.

02 Identify the Ethernet, TIB-IN and power connections.

03 Read the LED pattern and say what it means.

Result: a hardware record and one clear next action.



02

Commissioning the nanoBTS

Discovery and addressing, then telnet diagnostics and the BSC connection



Recovery and factory reset

Reset is a recovery option when the existing configuration is unusable.

Before reset

Record the current settings and identify the exact hardware revision.

Reset method

Use the correct TIB-IN reset plug and the model’s documented sequence.

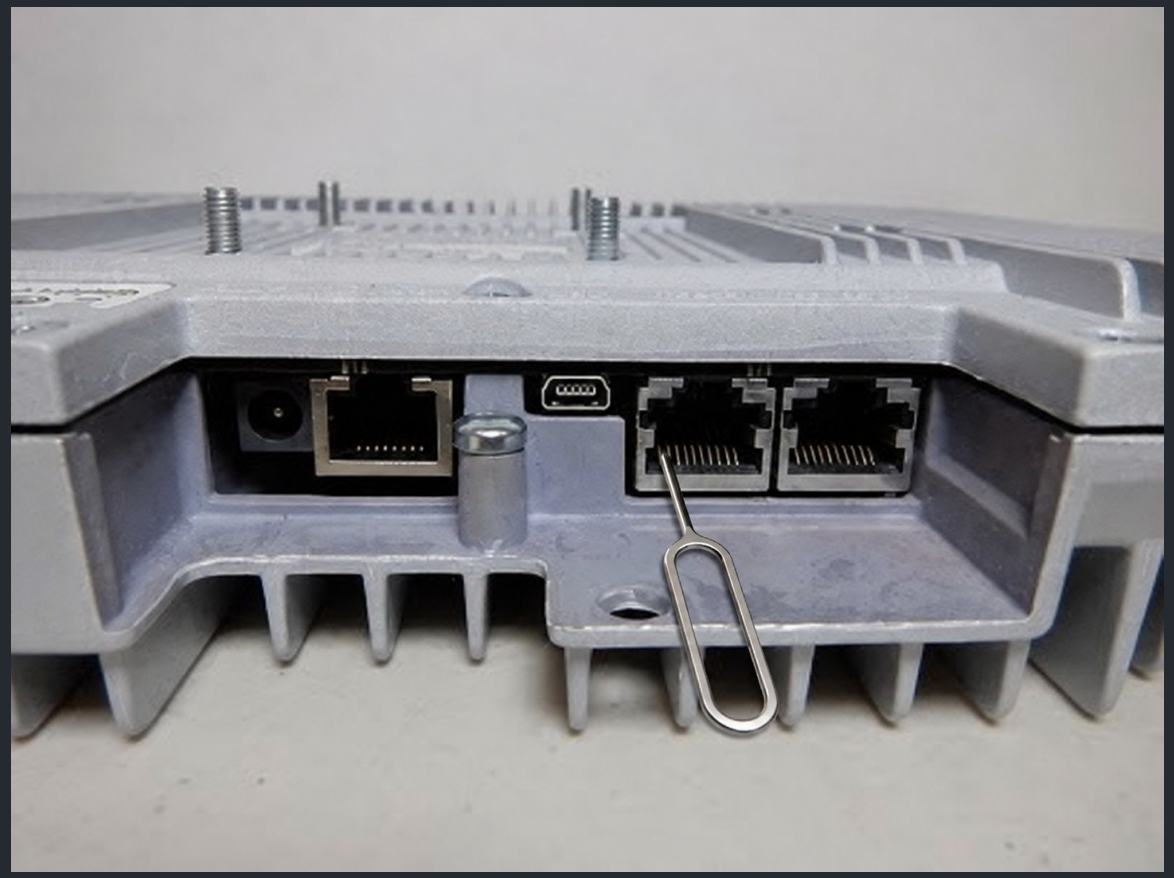
Confirmation

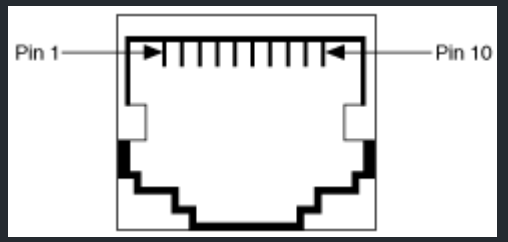
The fast red blink indicates that factory defaults have been applied.
Rediscover the unit afterwards.



Now for the reset

Bridge these two pins (9 & 10) and apply power





Rootworks

Discovering the BTS

Run discovery on the Ethernet segment that contains the unit.

```
sudo apt install abisip-find \
  osmo-bsc-ipaccess-utils
abisip-find --help
sudo abisip-find enp11s0
```

Record these fields

IP_Address and Unit_ID
Equipment / software version
Serial_Number

Alternative evidence

DHCP lease logs or an Ethernet capture.

Result: the discovered IP matches the unit you intend to configure.



One address plan for the walkthrough

These values must agree across the BTS and core.

Item	Course example	Used by
Lab host	10.0.0.251	BSC, RTP and SGSN listener
nanoBTS	10.0.0.100	Static management address
Unit ID	1/0/0	BTS commissioning and BSC
Network	001 / 01, GSMLab	BSC and MSC
APN / tunnel	internet / tun-osmo	Device and GGSN
Device IP pool	172.16.23.0/24	PDP contexts



Unit ID and OML destination

The BTS needs to know which BSC to connect to.

```
ipaccess-config -u 1/0/0 10.0.0.48
ipaccess-config -o 10.0.0.251 10.0.0.48
```

BSC configuration

bts 0
 ipa unit-id 1 0

What changed

Match the unit ID.
Point OML at 10.0.0.251.

Check each write’s response before continuing.



Static IP and restart

Keep a route back to the device while changing its address.

```
ipaccess-config -i \
  10.0.0.100/255.255.255.0 10.0.0.48

ipaccess-config -S static-ip \
  -U dhcp-enabled 10.0.0.48

ipaccess-config -r 10.0.0.48
```

After restart

Rediscover the unit.
Expect 10.0.0.100.

Telnet access

Next: enable the BTS CLI and build the ip.access client.



Result: the unit is reachable at the planned address.

Enabling nanoBTS telnet

Enable the CLI flag at the device’s current reachable address.

```
ipaccess-config -S cli-enabled \
  10.0.0.100

ipaccess-config -r 10.0.0.100
```

BTS restart

Wait for the unit to return at 10.0.0.100.

Client choice

Use ipaccess-telnet for the BTS diagnostic session.



Result: the BTS CLI is enabled for the telnet exercise.

Telnet client build prerequisites

Prepare the Debian or Ubuntu lab VM before building libtelnet.

```
sudo apt install \
  ca-certificates git build-essential \
  autoconf automake libtool pkg-config \
  libssl-dev zlib1g-dev
```

Package step

Install dependencies once on the prepared VM.

Build step

Run the following source build as your regular user.

Result: the build tools and development libraries are available.



Building and opening the telnet client

Build the client, then connect to the commissioned nanoBTS.

```
git clone \
  https://github.com/osmocom/libtelnet
cd libtelnet
autoreconf --force --install
./configure
make
./util/ipaccess-telnet 10.0.0.100
```

Build location

Work in your home or course directory.

Connection

Use the BTS address and expect a firmware-specific prompt.

Result: a diagnostic session on the BTS at 10.0.0.100.



Exercise 2: commissioning check

The unit is visible on Ethernet. Can it find its BSC?

01 Rediscover the unit and verify the planned IP and unit ID.

02 Confirm the primary OML destination is 10.0.0.251.

03 Open the BTS diagnostic session. Keep OML verification for core startup.

Result: addressing and identity agree. OML will be checked once the core runs.



03

Installing and configuring the core

A prepared package set and one consistent configuration



A repeatable software baseline

Use the same package snapshot for rehearsal and delivery.

Packages

Use the repository for the VM’s distribution and release. Freeze the course’s nightly package set after rehearsal.

Version record

Save the complete installed package inventory and snapshot the working VM.

Configuration

Install the course configuration files and check them against that package version.



Repository setup reference

Ubuntu 26.04 nightly example for the prepared class VM

```
wget \
 https://obs.osmocom.org/projects/osmocom/public_key
gpg --show-keys --with-fingerprint public_key
sudo install -Dm644 public_key \
 /usr/share/osmocom-keyring/osmocom.asc
osmo_base=https://downloads.osmocom.org/packages
osmo_repo="$osmo_base/osmocom:/nightly/xUbuntu_2604"
osmo_key=/usr/share/osmocom-keyring/osmocom.asc
printf 'deb [signed-by=%s] %s/ ./\n' \
 "$osmo_key" "$osmo_repo" | \
 sudo tee /etc/apt/sources.list.d/osmocom.list
sudo apt update
```



Use the repository for the installed OS release. Confirm the signing-key fingerprint.

Installing the core packages

The nanoBTS supplies the BTS software, so this path does not need OsmoTRX.

```
sudo apt update
sudo apt install \
  osmo-bsc osmo-stp osmo-hlr osmo-msc \
  osmo-mgw osmo-sgsn osmo-ggsn \
  abisip-find osmo-bsc-ipaccess-utils

dpkg-query -W > packages.txt
```

Before this step

Configure the matching repository and signing key.

After installation

Check for automatically started services before launching your own instances.

Result: the required binaries and discovery tools are installed.



Loading the course configuration

The examples are a lab baseline that you adapt to the actual host.

Source

Use the supplied course bundle or the recorded revision of mike-rootworks/osmocom-configs.

Copy

Back up existing files. Install the intended .cfg files into the chosen configuration directory.

Review

Check the host IP, unit ID, APN, radio settings and state paths before starting.



The network identity

The BSC and MSC must describe the same lab network.

```
network
 network country code 001
 mobile network code 01
 short name GSMLab
 long name GSMLab
 encryption a5 0
```

Configuration files

osmo-bsc.cfg
osmo-msc.cfg

Meaning

A5/0 disables GSM air encryption in this baseline. Application TLS is separate.

Check: PLMN and lab name agree in both files.



The radio settings that matter

Identity, channel selection and packet-radio mode

```
bts 0
 type nanobts
 band DCS1800
 ipa unit-id 1 0
 gprs mode egprs
 trx 0
  rf_locked 1
  arfcn 878
  max_power_red 20
```

Hardware identity

DCS1800 and unit ID 1/0/0 must match the physical unit.

Packet radio

Use gprs if the unit or firmware does not support egprs.



Keep the saved configuration locked so restart returns to a known state.

The timeslot plan

One carrier shares radio capacity between signalling, voice and packet data.

Timeslot	Source configuration	Purpose
0	CCCH+SDCCH4	Broadcast/access and signalling
1	SDCCH8	Dedicated signalling
2–3	TCH/F	Full-rate voice
4–6	TCH/F_PDCH	Dynamic voice or packet data
7	PDCH	Packet data



BSC reference: identity and packet path

Selected settings from the full course configuration

```
bts 0
 type nanobts
 band DCS1800
 ipa unit-id 1 0
 gprs mode egprs
 gprs cell bvci 24
 gprs nsei 24
 gprs nsvc 0 nsvci 24
 gprs nsvc 0 local udp port 23000
 gprs nsvc 0 remote udp port 23000
 gprs nsvc 0 remote ip 10.0.0.251
```



Keep BVCI, NSEI and NSVC consistent with the SGSN. Full file in presenter notes.

HLR reference: registration policy

Auto-created subscribers are an explicit choice in this lab.

```
hlr
 subscriber-create-on-demand 3 cs+ps
 store-imei
 gsup
  bind ip 127.0.0.1
 ussd route prefix *#100# internal own-msisdn
 ussd route prefix *#101# internal own-imsi
 ps
  pdp-profiles default
```



MSC check-imei-rqd early complements HLR store-imei. APN profiles remain in the full file.

Voice reference: control and media

MGCP and RTP use different addresses and purposes.

BSC MGW client

```
mgw 0
 remote-ip 127.0.0.1
 remote-port 2427
 local-port 2727
```

MGW service

```
mgcp
 bind ip 127.0.0.1
 bind port 2427
 rtp bind-ip 10.0.0.251
 rtp port-range 4002 16001
```



The course uses one MGW listener. Rehearse endpoint allocation and two-way audio.

Packet reference: SGSN and GGSN

The radio-facing Gb listener differs from the local GTP path.

SGSN selected lines

```
sgsn
 gtp local-ip 127.0.0.1
 ggsn 0 remote-ip 127.0.0.2
 authentication optional
 auth-policy remote
ns
 bind udp local
  listen 10.0.0.251 23000
```

GGSN selected lines

```
ggsn ggsn0
 gtp bind-ip 127.0.0.2
 apn internet
  tun-device tun-osmo
  no shutdown
```



Device pool: 172.16.23.0/24. Complete configuration files are in presenter notes.

MSC reference: switching and SMS

Subscriber attachment, local calling and message persistence

```
msc
 assign-tmsi
 check-imei-rqd early
 mncc internal
smsc
 database /var/lib/osmocom-lab/sms.db
 database delete-delivered 0
 database delete-expired 0
 queue max-failure 3
```



The full network and MGW sections are in presenter notes. Retention needs a deliberate policy.

Which address belongs where?

Separate the radio-facing listeners from the local control paths.

Interface	Course example	Why it matters
BTS control	BSC on 10.0.0.251	Matches the primary OML IP
Voice RTP	MGW on 10.0.0.251	Reachable from the nanoBTS
Packet Gb / NS	SGSN on 10.0.0.251:23000	Receives the PCU’s packets
Subscriber GSUP	HLR on 127.0.0.1:4222	MSC and SGSN use it
Packet GTP	SGSN 127.0.0.1 / GGSN 127.0.0.2	Core processes on this host
Voice MGCP	MGW on 127.0.0.1:2427	Controller-to-gateway control



APN, IP pool and DNS

GGSN allocates the device IP when it activates a PDP context.

```
apn internet
 gtpu-mode tun
 tun-device tun-osmo
 type-support v4
 ip prefix dynamic 172.16.23.0/24
 ip dns 0 10.0.0.251
 ip dns 1 10.0.0.251
 no shutdown
```

For the offline exercise

Both advertised resolvers point to the lab simulator.

Important distinction

The device pool is for PDP contexts. It is not a DHCP lease pool.



Result: the APN and resolver policy describe the intended test mode.

Exercise 3: spot the mismatch

The radio responds on Ethernet, but the BSC never accepts it.

01 nanoBTS reports Unit_ID = 21/0/0.

02 The BSC contains ipa unit-id 1 0.

03 Identify the mismatch and state the two values you would make consistent.

Explain the correction before changing either side.



04

Bringing the services to life

Core links first, then a controlled service test



Startup order

Use foreground terminals for the first rehearsal so errors stay visible.

1. **Subscriber and signalling services**
   Start HLR and STP.
2. **Media and switching**
   Start MGW, then MSC.
3. **Packet core**
   Start GGSN, then SGSN.
4. **Radio controller**
   Start BSC and inspect the BTS connection.

Example: sudo osmo-bsc -c /etc/osmocom/osmo-bsc.cfg



Checkpoint: the core is ready

A running process is only the first check.

Radio link

BSC shows working OML and RSL connections to the intended nanoBTS.

Core signalling

BSC and MSC complete their signalling association through STP.

Subscriber and packet paths

MSC and SGSN connect to HLR. GGSN creates tun-osmo.



RF control from the BSC

Unlock only the intended TRX for the exercise, then lock it again afterwards.

```
enable
configure terminal
network
bts 0
trx 0
rf_locked 0
```

BSC VTY

telnet 127.0.0.1 4242

End the exercise

Use the same path with rf_locked 1.



Check the reported BTS state and the expected cell on a test device.

Exercise 4: call and SMS

Two phones, two local numbers, two clear acceptance tests.

01 Register both test devices and record their assigned lab numbers.

02 Make a call. Confirm audio in both directions and a clean release.

03 Send “LAB-001” in each direction. Confirm it on the receiving device.

Result: registration, two-way voice and device-confirmed SMS.



The call connects. Nobody hears anything.

Where would you look first?

A The HLR subscriber number

B The MGW RTP address and media packets

C The GPRS APN

Discuss your reasoning before opening the captures.



05

GPRS, EDGE and the test endpoint

A PDP session, a resolver and an application exchange



Packet data in five checks

Move to the next layer only after you have evidence of the previous one.

1. **Radio resources**
   Packet-capable firmware and usable PDCH timeslots.
2. **GPRS attachment**
   The device appears in the SGSN.
3. **PDP context**
   The requested APN resolves to the GGSN and allocates an IP.
4. **DNS**
   The device sends a query to the intended resolver.
5. **Application traffic**
   A controlled endpoint receives the expected request.



Installing and preparing INetSim

Use the package configuration as the base for the lab copy.

```
sudo apt install inetsim dnsutils

mkdir -p inetsim-lab
cp /etc/inetsim/inetsim.conf \
  inetsim-lab/inetsim.conf

nano inetsim-lab/inetsim.conf
```

Prepared lab VM

Install packages before disconnecting the upstream link.

Working copy

Retain packaged paths and users. Apply the next slide’s settings.



Use one INetSim instance. Stop any packaged instance before starting the lab copy.

INetSim services and binding

Enable DNS and web services on the lab host at 10.0.0.251.

```
service_bind_address 10.0.0.251
dns_default_ip 10.0.0.251
start_service dns
start_service http
start_service https
create_reports YES
```

Working configuration

Remove other start_service entries. Keep the package defaults.

Expected behaviour

DNS returns the simulator address. HTTP reaches the simulator.



Check: no other resolver or web service occupies these listener ports.

Starting and checking INetSim

Run the simulator in one terminal. Check it from another.

```
sudo inetsim --config \
  "$PWD/inetsim-lab/inetsim.conf"

# A second terminal on the lab host
dig @10.0.0.251 example.com A
curl --noproxy '*' http://10.0.0.251/
```



Expect DNS to return 10.0.0.251 and HTTP to return the simulator’s page.

Advertising the simulator DNS

Edit the internet APN in the active GGSN configuration.

```
# Within apn internet in osmo-ggsn.cfg
  ip dns 0 10.0.0.251
  ip dns 1 10.0.0.251

# Stop the foreground GGSN, then restart it
sudo osmo-ggsn -c /etc/osmocom/osmo-ggsn.cfg
```



Reconnect packet data so the device receives the new DNS options.

Configuration paths depend on the installer

Find the active -c argument or the container’s mounted file.

Deployment	GGSN configuration	Restart method
Native walkthrough	/etc/osmocom/osmo-ggsn.cfg	Restart the foreground process
Course installer	/etc/osmocom-lab/osmo-ggsn.cfg	Installer service unit
Docker installer	/opt/osmocom-lab-docker/config/osmo-ggsn.cfg	Compose osmo-ggsn service
Snap Docker	/var/snap/docker/common/ osmocom-lab-docker/config/osmo-ggsn.cfg	Use the actual compose file



Offline traffic rules: simulator access

Native lab VM: create these chains once, then add the permits.

```
sudo iptables -N OSMOLAB-IN
sudo iptables -N OSMOLAB-FWD

sudo iptables -A OSMOLAB-IN \
 -s 172.16.23.0/24 -d 10.0.0.251 \
 -p udp --dport 53 -j ACCEPT
sudo iptables -A OSMOLAB-IN \
 -s 172.16.23.0/24 -d 10.0.0.251 \
 -p tcp -m multiport --dports 53,80,443 -j ACCEPT
```



These rules allow the mobile pool to use INetSim DNS, HTTP and HTTPS.

Offline traffic rules: drops and hooks

Complete the chains before attaching them to tunnel traffic.

```
sudo iptables -A OSMOLAB-IN -j DROP
sudo iptables -A OSMOLAB-FWD -j DROP

sudo iptables -I INPUT 1 -i tun-osmo \
 -j OSMOLAB-IN
sudo iptables -I FORWARD 1 -i tun-osmo \
 -j OSMOLAB-FWD

sudo iptables -L OSMOLAB-IN -nv
sudo iptables -L OSMOLAB-FWD -nv
```



Reconnect the device, make one request and watch the rule counters.

Exercise 5: follow one data request

Choose a device action with an observable result.

01 Set APN internet and request packet data. Record the allocated IP.

02 Trigger one known HTTP request or the device’s documented check-in.

03 Compare the simulator report with the IP capture and timestamps.

Result: a request you can explain from the device to the endpoint.



Capturing the evidence

Ethernet shows the core traffic. The TUN interface shows decapsulated IP.

```
sudo tcpdump -i enp11s0 -s 0 \
  -w bts-ethernet.pcap

sudo tcpdump -i lo -s 0 \
  -w core-loopback.pcap

sudo tcpdump -i tun-osmo -s 0 \
  -w device-ip.pcap
```

Capture only what is needed

Run captures in separate terminals. Stop after the test transaction.

Correlate the result

Device action, service logs, packet timestamps and the test token.



A packet capture does not decrypt application TLS by itself.

Osmocom-lab usage:

Command	What it does
sudo osmocom-lab start	Starts the lab core and its firewall rules.
sudo osmocom-lab stop	Stops the lab services and removes their firewall rules.
sudo osmocom-lab status	Shows service status and listening network ports.
sudo osmocom-lab rf-enable	Unlocks the configured radio carrier.
sudo osmocom-lab rf-disable	Locks the configured radio carrier.
sudo osmocom-lab firewall-up	Applies the lab’s firewall and internet-sharing rules.
sudo osmocom-lab firewall-down	Removes the lab’s own firewall rules.



06

SMS from Python

A repeatable stimulus and a result you can verify



The SMPP interface

Python connects to the MSC's SMSC. GPRS is not involved.

```
smpp
 local-tcp-ip 127.0.0.1 2775
 system-id GSMLab
 policy closed
 no smpp-first
 esme sms-client
  no password
  no alert-notifications
```

Course baseline

Run Python on the core host. It binds as sms-client.

A separate setting

SMPP policy closed controls ESME access, not HLR subscriber registration.

Check that the installed MSC recognises the smpp configuration node.



Sending a test SMS

Use the course script’s actual arguments.

```
python3 -m venv .venv-sms
.venv-sms/bin/pip install smpplib

.venv-sms/bin/python scripts/send-sms.py \
  123 9000 127.0.0.1 "LAB-002"
```

Argument order

Recipient, sender, server IP, message.

Replace 123

Use the actual attached device’s lab MSISDN.

Result: “LAB-002” appears on the receiving device.



Python says “accepted”. Did the device receive it?

What evidence would let you answer confidently?

A A successful SMPP submit response

B The unique test token on the handset or modem

C The Python process exiting without an error

Choose the evidence, then explain what the other two results prove.



07

Device identification and SMS evidence

Finding the APN, listing subscribers and reviewing messages across the lab



Finding the APN in an active session

The SGSN links the packet session to the device’s IMSI.

```
telnet 127.0.0.1 4245
OsmoSGSN> show pdp-context all
OsmoSGSN> show mm-context all pdp

# Illustrative output, not a recorded device
PDP Context IMSI: 001010000000001
  APN: internet
  PDP Address: IPv4 172.16.23.2
```

Record the IMSI, session APN and allocated IP together.



The APN requested by the device

Capture a fresh activation attempt, including an attempt that fails.

```
sudo tcpdump -ni enp11s0 -s0 -U \
 -w apn-discovery.pcap udp port 23000

# Wireshark display filter on the Gb capture
gsm_a.gm.sm.apn

# For a separate SGSN/GGSN signalling capture
gtp.apn
```

Reconnect packet data. Read the APN in Activate PDP Context Request.



Enumerating the HLR

List all known subscribers, then inspect the connecting IMSI.

```
telnet 127.0.0.1 4258
OsmoHLR> show subscribers all
OsmoHLR> show subscribers last-seen
OsmoHLR> subscriber imsi 001010000000001 show

# Current circuit-switched cache: a separate VTY
telnet 127.0.0.1 4254
OsmoMSC> show subscriber cache
```



Use the HLR’s IMSI/MSISDN map to identify SMS source and destination numbers.

Retaining SMS for the lab run

Set retention before exchanging the test messages.

```
telnet 127.0.0.1 4254
OsmoMSC> enable
OsmoMSC# configure terminal
OsmoMSC(config)# smsc
OsmoMSC(config-smsc)# database delete-delivered 0
OsmoMSC(config-smsc)# database delete-expired 0
OsmoMSC(config-smsc)# end
OsmoMSC# show running-config
```



Record the smsc database path. Retention applies to messages stored from now on.

Messages for all devices in the SMS database

Set SMS_DB to the active smsc database path shown by the MSC.

```
SMS_DB=/path/from/smsc/database

sqlite3 -readonly "$SMS_DB" <<'SQL'
.headers on
.mode csv
.once sms-all.csv
SELECT id, created, sent, src_addr, dest_addr,
       deliver_attempts, data_coding_scheme, text,
       hex(user_data) AS payload_hex
FROM SMS ORDER BY id;
SQL
```

The CSV includes every retained row. Match src_addr and dest_addr to the HLR list.



Capturing SMS across the lab

Start both captures before any device sends or receives a test SMS.

```
# Terminal 1: nanoBTS signalling on Ethernet
sudo tcpdump -ni enp11s0 -s0 -U \
 -w sms-bts.pcap

# Terminal 2: native core signalling on loopback
sudo tcpdump -ni lo -s0 -U \
 -w sms-core.pcap

# Wireshark display filter
gsm_sms
```



Use the protocol filter across all devices. Stop each capture with Ctrl-C.

Reading and exporting the SMS exchange

MO means device to network. MT means network to device.

```
tshark -r sms-core.pcap -Y gsm_sms \
 -T fields -E header=y -E separator=, \
 -E quote=d -E occurrence=a \
 -e frame.time_epoch -e gsm_sms.tp-oa \
 -e gsm_sms.tp-da -e gsm_sms.sms_text \
 > sms-captured.csv
```



In Wireshark, inspect SMS-SUBMIT or SMS-DELIVER and the matching RP-ACK/RP-ERROR.

Exercise 6: one device, complete evidence

Use the subscriber list to link packet data and SMS to the same device.

01 Reconnect packet data and record the requested APN plus IMSI.

02 Send one SMS in each direction. Match the numbers to the HLR.

03 Compare the retained SMS rows with the capture and delivery result.

Result: one evidence record per device, with gaps and failures explained.



An optional real Internet baseline

Change test mode deliberately once the offline path works.

Routing

Select the intended uplink. Enable IPv4 forwarding and scoped NAT for the device pool.

DNS

Advertise the intended resolver and verify the device uses it.

Comparison

Capture one successful direct exchange and record what changes from the offline run.



Closing the lab cleanly

Leave a setup that you can repeat next time.

Radio and processes

Lock RF, power down the BTS and stop the core instances you started.

Test services

Stop the simulator and captures. Restore any forwarding or firewall changes made for the test.

Evidence and baseline

Save the useful captures, config revision and version record. Preserve a clean starting snapshot.



What can you now explain?

Use the lab results to tell the story.

01 How did the nanoBTS find the BSC?

02 Which paths carried call audio, SMS and IP data?

03 What proves success, and what would you inspect if the test failed?



Questions and discussion

mike@rootworks.co.uk / rootworks.co.uk

08

Legal reference

UK spectrum use and responsibilities for client tests

UK private cellular networks

Radio use needs an Ofcom licence or a specific statutory exemption.

A Faraday chamber alone does not establish an exemption

Wireless Telegraphy Act 2006, section 8 applies to establishing a station and installing or using radio apparatus.

The authorisation must cover the actual use

Shared Access supports private networks. For testing on other bands, ask Ofcom about Innovation and Trial licensing.

Breaches can lead to enforcement

Unlicensed use or breach of licence conditions can be a criminal offence, with fines and possible equipment seizure.

Ofcom radio-use rules and licensing guidance. General information, not legal advice.

Faraday chamber: remaining risks

Shielding performance and operating procedures need evidence.

RF leakage

Doors, seals and cable feedthroughs can leak. Check containment across every test band at the maximum intended power.

Unintended handset connections

Use controlled test SIMs and authorised devices. Leakage or a configuration error can expose nearby phones to the cell.

RF exposure inside the chamber

Assess exposure for staff and visitors, including people with implanted medical devices. Shielding does not prove safe exposure.

Recommended controls: door interlock, accessible RF stop and retained test records.

Controls are recommendations for the lab. Ofcom conditions and HSE exposure duties still apply.



Client responsibilities and limits

Agree the test scope and who controls the equipment before RF starts.

Permission and responsibility

Document authority for devices, SIMs and tests. The licence holder remains responsible when clients control the radio equipment.

Privacy and communications

Identifiers and logs may be personal data. Establish a lawful basis, explain recording, restrict access and set deletion periods.

Interception and interference

A spectrum licence does not authorise unlawful interception. Intentional interference needs separate legal authority.

Do not rely on the isolated lab network for 999/112. Provide a separate emergency contact method.

Any public service or external interconnection needs a separate review of telecoms obligations.



Applying for shared 1800 MHz

Shared Access Low Power is the usual starting point for a small lab.

PAIRED SPECTRUM

1781.7–1785 MHz uplink

1876.7–1880 MHz downlink

£80/year per Low Power area. 50 m radius.
Base station limit: 24 dBm EIRP per carrier up to 3 MHz.

1 Register at Ofcom’s licensing portal and choose Shared Access.
2 Select Low Power and 1800 MHz. Supply the site and RF details.
3 Ofcom assesses availability and interference, then invoices you.
4 Pay and wait for the issued licence before installing or using RF.

ofcom.org.uk/spectrum/radio-equipment/licensing-portals / spectrum.licensing@ofcom.org.uk



Thank you

mike@rootworks.co.uk

